

```
!pip install nltk
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-package  
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packag  
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packa  
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/d  
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-package
```

```
import nltk
```

```
nltk.download('punkt')  
nltk.download('punkt_tab')  
nltk.download('stopwords')  
nltk.download('wordnet')  
nltk.download('averaged_perceptron_tagger_eng')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...  
[nltk_data]   Package punkt is already up-to-date!  
[nltk_data] Downloading package punkt_tab to /root/nltk_data...  
[nltk_data]   Package punkt_tab is already up-to-date!  
[nltk_data] Downloading package stopwords to /root/nltk_data...  
[nltk_data]   Package stopwords is already up-to-date!  
[nltk_data] Downloading package wordnet to /root/nltk_data...  
[nltk_data]   Package wordnet is already up-to-date!  
[nltk_data] Downloading package averaged_perceptron_tagger_eng to  
[nltk_data]   /root/nltk_data...  
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.  
True
```

```
from nltk.tokenize import word_tokenize, sent_tokenize
```

```
text = "Tokenization is the first step in text analytics."  
print(sent_tokenize(text))  
print(word_tokenize(text))
```

```
['Tokenization is the first step in text analytics.']  
['Tokenization', 'is', 'the', 'first', 'step', 'in', 'text', 'analytics', '.']
```

```
from nltk.corpus import stopwords
import re

text = "How to remove stop words with NLTK library in Python?"

stop_words = set(stopwords.words("english"))

text = re.sub('[^a-zA-Z]', ' ', text)
tokens = word_tokenize(text.lower())

filtered = [w for w in tokens if w not in stop_words]

print(filtered)
```

```
['remove', 'stop', 'words', 'nltk', 'library', 'python']
```

```
from nltk.corpus import stopwords
import re

text = "How to remove stop words with NLTK library in Python?"

stop_words = set(stopwords.words("english"))

text = re.sub('[^a-zA-Z]', ' ', text)
tokens = word_tokenize(text.lower())

filtered = [w for w in tokens if w not in stop_words]

print(filtered)
```

```
['remove', 'stop', 'words', 'nltk', 'library', 'python']
```

```
from nltk.stem import PorterStemmer

words = ["studies", "studying", "cries", "cry"]

ps = PorterStemmer()

for w in words:
    print(ps.stem(w))
```

```
studi
studi
cri
cri
```

```

from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()

words = ["studies", "studying", "cries", "cry"]

for w in words:
    print(lemmatizer.lemmatize(w))

```

```

study
studying
cry
cry

```

```

import nltk
from nltk.tokenize import word_tokenize

sentence = "The pink sweater fit her perfectly"

words = word_tokenize(sentence)

print(nltk.pos_tag(words))

```

```

[('The', 'DT'), ('pink', 'NN'), ('sweater', 'NN'), ('fit', 'VBP'), ('her', 'P

```

```

import nltk
nltk.download("averaged_perceptron_tagger_eng")
from nltk import pos_tag
token=word_tokenize(sentence)
cleaned_token=[]
for word in token:
    if word not in stop_words:
        cleaned_token.append(word)
tagged=pos_tag(cleaned_token)
print(tagged)

```

```

[('The', 'DT'), ('pink', 'NN'), ('sweater', 'NN'), ('fit', 'VBP'), ('perfectl
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data] date!

```

```

import math
import pandas as pd

documentA = "Jupiter is the largest Planet"
documentB = "Mars is the fourth planet from the Sun"

# BoW
bagA = documentA.split()
bagB = documentB.split()

# Unique words

```

```

uniqueWords = set(bagA).union(set(bagB))

# Word count
numA = dict.fromkeys(uniqueWords, 0)
for word in bagA:
    numA[word] += 1

numB = dict.fromkeys(uniqueWords, 0)
for word in bagB:
    numB[word] += 1

# TF
def computeTF(wordDict, bag):
    tf = {}
    total = len(bag)
    for word, count in wordDict.items():
        tf[word] = count / total
    return tf

tfA = computeTF(numA, bagA)
tfB = computeTF(numB, bagB)

# IDF
def computeIDF(docs):
    N = len(docs)
    idf = dict.fromkeys(docs[0].keys(), 0)

    for doc in docs:
        for word, val in doc.items():
            if val > 0:
                idf[word] += 1

    for word in idf:
        idf[word] = math.log(N / float(idf[word]))

    return idf

idf = computeIDF([numA, numB])

# TF-IDF
def computeTFIDF(tf, idf):
    tfidf = {}
    for word, val in tf.items():
        tfidf[word] = val * idf[word]
    return tfidf

tfidfA = computeTFIDF(tfA, idf)
tfidfB = computeTFIDF(tfB, idf)

df = pd.DataFrame([tfidfA, tfidfB])
print(df)

```

	the	Mars	is	Jupiter	fourth	Sun	Planet	planet	\
0	0.0	0.000000	0.0	0.138629	0.000000	0.000000	0.138629	0.000000	
1	0.0	0.086643	0.0	0.000000	0.086643	0.086643	0.000000	0.086643	

```
      largest      from
0  0.138629  0.000000
1  0.000000  0.086643
```

```
from sklearn.feature_extraction.text import TfidfVectorizer

docs = [
    "Jupiter is the largest planet",
    "Mars is the fourth planet from the sun"
]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(docs)

print(vectorizer.get_feature_names_out())
print(X.toarray())
```

```
['fourth' 'from' 'is' 'jupiter' 'largest' 'mars' 'planet' 'sun' 'the']
[[0.         0.         0.37930349 0.53309782 0.53309782 0.
  0.37930349 0.         0.37930349]
 [0.37695709 0.37695709 0.26820807 0.         0.         0.37695709
  0.26820807 0.37695709 0.53641614]]
```

Start coding or [generate](#) with AI.